Abstract

In-depth analysis and investigation of a Virtual Mouse and Keyboard system operated by hand gestures are the focus of this research paper. In the field of human-computer interaction, the development of input devices needs to be user-friendly and effective. This paper provides a new approach to build a virtual mouse using patagium, OpenCV, Media pipe and other cutting-edge technologies, as well as mathematical techniques. The goal is to offer a new way of operating a computer interface that will be especially useful for users who have difficulty moving around or need more control over the computer interface. Media Pipe is used for hand-finger following and allows for real-time positioning of signals as well as development designs. OpenCV can be used as a comprehensive multipurpose system for picture preparation and control. Patagium is a Python-based robotization module that automatically changes over hand gestures to simulate mouse activities on the screen. This examination highlights the potential to combine these innovations to create a more comprehensive, open computing environment that can cater to a wider range of clients and open up new conceivable outcomes for human-computer interactions.

Contents

LIST OF FIGURES……

1.INTRODUCTION……

1.1 Motivation……..

1.2 Problem Definition…….

1.3 Objectives…….....

1.4 Scope…….....

1.5 Organization of Report……

2 LITERATURE SURVEY……

2.1 Existing System…….....

2.2 Research Papers Reviewed…….....

2.3 Summary…….

3SOFTWARE REQUIREMENTS SPECIFICATION.....

3.1 Factional Requirements……..

3.2 Non-Functional Requirements…….....

3.3 System Requirements……..

3.4 Hardware Software Specifications……..

4.SYSTEM DESIGN…….....

4.1 Architecture Diagram……..

4.2 Data Flow Diagram (DFD)…….

4.3 Entity Relationship Diagram (ERD)……

4.4 UML Diagram…….

5.SYSTEM IMPLIMENTATION……....

5.1 Modules……...

5.2 Algorithms and Methodology…….

5.3 Results and Discussion……...

5.4 Coding……

5.5 Output…….....

6 CONCLUSION AND FUTERE SCOPE……

6.1 Conclusion…….

6.2 Future Enhancements……

LIST OF FIGURES

Fig 4.1 System Architecture…… 19

Fig 5.1 Output…… 33

LIST OF TABLES

Table 2.1 Research Paper Survey ……. 12

CHAPTER 1
INTRODUCTION

INTRODUCTION

1.1 Motivation: -

Virtual Mouse Hand Gestures technology revolutionizes computer interaction by harnessing the capabilities of Media Pipe and OpenCV, two powerful computer vision libraries. Media Pipe, developed by Google, provides accurate hand landmark detection, enabling the identification of various hand gestures in real-time. OpenCV complements Media Pipe by processing the data, refining gesture recognition through noise reduction algorithms and enhancing accuracy. The detected gestures are mapped to specific mouse actions and interpreted using mathematical algorithms. Trigonometric calculations convert hand positions into precise screen coordinates, ensuring accurate mapping of gestures to cursor movements. These interpreted gestures are translated into mouse actions using libraries like Patagium, enabling users to interact with applications, games, and graphical interfaces seamlessly. This technology has significant applications in accessibility, empowering individuals with mobility impairments, and offers gesture customization, enhancing user experience. Furthermore, it finds applications in gaming, virtual reality environments, and interactive multimedia systems, enhancing user immersion and engagement. By combining Media Pipe and OpenCV, developers create intuitive and hands-free methods of interacting with digital interfaces, making computing more accessible and user-friendly.

1.2 Problem Definition: -

Traditional physical keyboards require direct contact, occupy physical space, and can be inconvenient or unhygienic in certain environments. In scenarios such as virtual reality systems, public kiosks, medical settings, or for users with mobility limitations, the use of conventional keyboards becomes inefficient or impractical. Touchscreens, while reducing mechanical dependency, still require surface contact and lack tactile flexibility.

The problem is to design and develop an air-based keyboard system that enables users to input text by detecting finger movements or gestures in mid-air without the need for a physical keyboard. The system must accurately recognize intended keystrokes, provide real-time response, minimize input errors, and be comfortable for prolonged use.

Key challenges include reliable gesture detection, precise mapping of air gestures to keyboard characters, minimizing latency, reducing false inputs, and ensuring usability across different users and environments.

The goal is to create a cost-effective, portable, and user-friendly air-based keyboard that can serve as an alternative input method for modern human-computer interaction systems.

1.3 Objectives: -

The main objectives of this project are as follows:

1. To design and develop an air-based keyboard that allows text input without physical contact.

2.To detect and track finger movements or hand gestures accurately in mid-air.

3.To map detected gestures to corresponding keyboard characters in real time.

4. To minimize input latency and improve typing accuracy.

5. To reduce hygiene issues associated with shared physical keyboards.

6. To provide an intuitive and user-friendly interface for efficient typing.

7. To ensure portability and compatibility with different computing devices.

8.To explore the use of sensors or vision-based technology for gesture recognition.

1.4 Scope: -

The scope of this project includes the design and implementation of an air-based keyboard system that enables users to input text through mid-air hand or finger gestures without the use of a physical keyboard. The system focuses on real-time gesture detection, tracking, and interpretation to generate accurate keystroke inputs.

The project covers the use of sensors or camera-based technologies to capture hand movements, processing algorithms to recognize gestures, and software modules to map these gestures to predefined keyboard layouts. The scope also includes basic user interface feedback to indicate key selection and input confirmation.

The system is intended for controlled environments and supports basic alphanumeric characters and common keyboard functions. Advanced features such as full multitouch simulation, haptic feedback, and large-scale language prediction may be considered beyond the current scope.

The project does not aim to completely replace traditional keyboards but serves as an alternative input method for applications such as virtual reality, public interfaces, assistive technologies, and touch-free computing environments.

1.5 Organization of Report: -

A report on an Air-Based Keyboard follows this streamlined technical structure:

1. Abstract: Summary of the gesture-recognition technology and key findings.

2. Introduction: Project goals, such as improving accessibility or hygiene in medical settings.

3. **System Architecture**:

1. Hardware: Sensors (Leap Motion, cameras) and microcontrollers like Arduino.

2. Software: Image processing modules and character recognition algorithms.

3. Methodology: Data collection of hand gestures and training via Machine Learning.

4.Results: Data on typing accuracy and WPM.

5.Conclusion: Final assessment and future scope.

CHAPTER 2

LITERATURE SURVEY

LITERATURE SURVEY

2.1 Existing Systems: -

"Air-based keyboard" systems primarily consist of wearable gesture controllers or optical motion-tracking software that translates mid-air hand movements into digital text. In a different context, the term also refers to the melodica, a physical instrument powered by human breath. These devices use motion sensors (IMUs) like gyroscopes and accelerometers to track hand orientation and finger strikes without needing a flat surface commercially available wearable that fits over the fingers. It detects taps on any surface or mid-air gestures using a 6-axis gyroscope and accelerometers. It connects via Bluetooth to control VR headsets, smartphones, and Ecoservice's like thevetin or Ruha are handheld remotes for Smart TVs and Android boxes. They use 6-axis motion sensing to move a cursory waving the device in the air and typically feature a full QWERTY keyboard on the reverse side.

2.2 Research Papers Reviewed: -

To understand the current progress in counterfeit product detection, several research papers were reviewed. The most relevant ones are summarized below:

- AIR Virtual Keyboard (Aman Ahsan Khan et al., 2026) – This paper presents an air-based virtual keyboard using hand gestures, implemented with Python, OpenCV, and Media pipe. The system detects fingertips in real time, allowing touchless typing. It is innovative and user-friendly but depends heavily on camera quality and lighting, and complex gestures may reduce accuracy.

- AI Virtual Keyboard for Typing (Akshay Krishnan et al., 2024) – This research develops an AI-based virtual keyboard using Cone and Media pipe. It supports customizable gestures, portable use, and provides a hygienic interface. The system is computationally intensive and requires good lighting and possibly a GPU for smooth operation.

- Webcam-Based Virtual Keyboard (Vaslav Livada et al., 2018) – This study implements a virtual keyboard using a standard webcam with skin colour segmentation and template matching. It allows touchless typing but is sensitive to background and lighting conditions, which can affect accuracy.

- Virtual Keyboards with Deep Learning Gesture Recognition (Tae-Ho Lee et al., 2022) – This paper uses deep learning for real-time gesture recognition to improve virtual typing. It is robust and accurate but needs large datasets and high-end hardware, making it computationally heavy.

- Ambidextrous Virtual Keyboard (Tae-Ho Lee & Hyuk-Jae Lee, 2018) – This paper proposes a layout that supports typing with both hands using finger gesture recognition.

It improves typing flexibility, but complex gestures may slow down typing speed, and fast typing may reduce accuracy.

2.3 Summary

Sr.no	Title	Author & Year	Summary	Limitation
1	ON AIR Virtual Keyboard	Aman Ahsan Khan et al., 2026	Air-based virtual keyboard using hand gestures; Python, OpenCV, Media pipe; touchless typing interface.	Dependent on camera and lighting; gestures may reduce accuracy; minor latency.
2	AI Virtual Keyboard for Typing	Akshay Krishnan et al., 2024	AI-based virtual keyboard using Cone & Media pipe; customizable gestures; portable and hygienic.	Light and camera quality dependent; computationally intensive; may require GPU.
3	Virtual Keyboards with Deep Learning Gesture Recognition	Tae-Ho Lee et al., 2022	Deep learning-based real-time gesture recognition for typing.	Requires large datasets; computationally heavy; high-end hardware needed.
4	Webcam-Based Virtual Keyboard	Casal Livada et al., 2018	Uses webcam with skin segmentation and template matching for typing.	Accuracy affected by lighting and background; gesture detection limited.
5	Ambidextrous Virtual Keyboard	Tae-Ho Lee & Hyuk-Jae Lee, 2018	Layout supporting both hands; finger gesture recognition.	Complex gestures reduce speed; accuracy limited for fast typing.

Table 2.1 Research Paper Survey

CHAPTER 3

SOFTWARE REQUIREMENT SPECIFICATION

3.1 Functional Requirements

A functional requirement describes what the system should do — the specific features and functions the software must have to meet the user's needs. For an air-based virtual keyboard, the functional requirements define how the keyboard will detect, interpret, and respond to hand gestures.

1. Hand Detection

The system must identify the user's hand using a camera or webcam. This is the first step, as the keyboard cannot function without knowing where the user's hand is. Techniques like Cone or Media pipe track the hand in real time by analysing video frames.

2. Gesture Recognition

Once the hand is detected, the system recognizes gestures — like finger taps, swipes, or pinches — that correspond to pressing keys. This involves interpreting the hand's landmarks (like fingertips) and movements using computer vision algorithms or AI models.

3. Virtual Key Mapping

The recognized gesture is then mapped to a virtual key on the keyboard. For example, a tap of the index finger in a specific area corresponds to pressing the "A" key. This mapping ensures gestures translate into meaningful input.

4. Text Display

The system should display the typed characters in a text box on the screen. This provides immediate feedback to the user, confirming that their gestures are being recognized correctly.

5. Multi-language Support

The keyboard may support typing in different languages. This involves changing key layouts and characters according to user preferences, enabling flexibility for users worldwide.

6. Real-time Feedback

To ensure smooth operation, the system should show FPS (Frames Per Second) and recognition status. This helps users know if their gestures are being detected accurately and whether the system is responsive.

7. Customization

Users should be able to modify the keyboard's appearance, layout, and gestures. This enhances usability and comfort, allowing a personalized experience.

8. Click Simulation

The system must interpret specific gestures (like pinching or tapping in the air) as key presses. This allows interaction without touching a physical keyboard.

9. Error Handling

The software should alert users if gestures are not detected, ambiguous, or if the hand moves out of the camera's frame. This ensures the system is user-friendly and avoids confusion.

10. Compatibility

The keyboard must work across multiple devices like desktops, laptops, or tablets. This ensures the system is versatile and can be widely used.

3.2 Non-Functional Requirements

While functional requirements define *what the system does*, non-functional requirements describe *how well the system performs those functions*. They focus on the quality, usability, performance, and constraints of the system.

1. Performance
The system should respond instantly to hand gestures, with minimal delay (latency). Real-time processing ensures typing is smooth and natural, without lag.

2. Accuracy
The virtual keyboard must accurately detect hand movements and gestures. Misinterpretation can lead to wrong characters being typed. High accuracy is crucial for usability.

3. Reliability
The system should work consistently under various conditions — different lighting, backgrounds, or hand sizes. It should not crash or fail frequently.

4. Usability
The keyboard must be easy to use and intuitive. Users should learn it quickly without training. The interface should provide visual feedback, like highlighting keys when detected, to guide the user.

5. Portability
The software should run on multiple devices (laptops, desktops, tablets) and operating systems without requiring major changes. This increases accessibility.

6. Security
Since the system may capture video data, it should ensure privacy. The camera feed must not be stored unnecessarily or transmitted insecurely.

7. Scalability
The software should support future enhancements, like additional gestures, multi-language support, or integration with VR/AR devices, without major redesign.

8. Maintainability
The system should be easy to update or fix. Well-structured code and modular design make future improvements or bug fixes simpler.

9. Flexibility
Users should be able to customize the keyboard layout, colours, and gestures. The software should adapt to different user preferences and requirements.

10. Environmental Constraints
The keyboard should work under normal lighting conditions and with standard webcams. It should not require specialized hardware, making it accessible to a wide range of users.

3.3 SYSTEM REQUIREMENTS

Software Requirements: -

1. Operating System: Windows / Linux / macOS
2. Programming Language: Python / C++ / Java
3. Development Tools: IDE (VS Code / PyCharm / Eclipse)
4. Libraries & Frameworks: Computer Vision Library (OpenCV)Machine Learning Framework (TensorFlow / Porch – optional) Gesture Recognition Library
5. Drivers & APIs: Camera or sensor drivers input device APIs
6. Database (Optional): SQLite / Local storage for user settings
7. UI Software: GUI framework (Qt / Tainter / JavaFX)
8. Testing Tools: Debugging and performance testing tools
9. Security Software: Data encryption for stored user preferences

Hardware Requirements

1.Camera / Sensor: -HD webcam / IR camera / depth sensor Minimum resolution: 720p

2.Processing Unit: -PC / Laptop / Embedded system Minimum Dual-Core processor

3. Memory: -Minimum 4 GB RAM

4. Storage: - Minimum 10 GB free disk space

5. Display Unit: - Monitor / Screen for visual feedback

6. Input Devices: - Mouse (for initial setup)

7. Connectivity: - USB port / Wireless interface

8.Power Supply: -Stable power source

9. Mounting Stand: - Camera stand or fixed mount for proper alignment

3.4 Hardware and Software Specifications

Component / Software	Specification
Camera / Sensor	HD Webcam / IR Camera (≥ 720p, 30 FPS)
Processor	Dual-Core Processor (Intel i3 or equivalent)
RAM	Minimum 4 GB
Storage	Minimum 10 GB free disk space
Display Unit	Monitor / Screen
Connectivity	USB 2.0 / USB 3.0 / Wireless
Power Supply	Stable power source
Mounting Equipment	Camera stand or fixed mount
Operating System	Windows 10 / Linux / macOS
Programming Language Python	Python / C++ / Java
Development Environment	VS Code / PyCharm / Eclipse
Libraries & Frameworks	OpenCV, TensorFlow / Porch (optional)
Device Drivers	Camera / Sensor drivers
User Interface	Qt / Tkinter / JavaFX
Database (Optional)	SQLite / Local Storage
Testing Tools	Debugging & performance tools
Security	Data protection for user settings

CHAPTER 4

SYSTEM DESIGN

4.1 Architecture Diagram

An air-based keyboard (also called a laser keyboard or virtual projection keyboard) projects a keyboard layout onto a flat surface and detects finger movements to register keystrokes.

Below is a clear explanation of the architecture diagram components and how they connect, which you can use to draw or understand the system design.

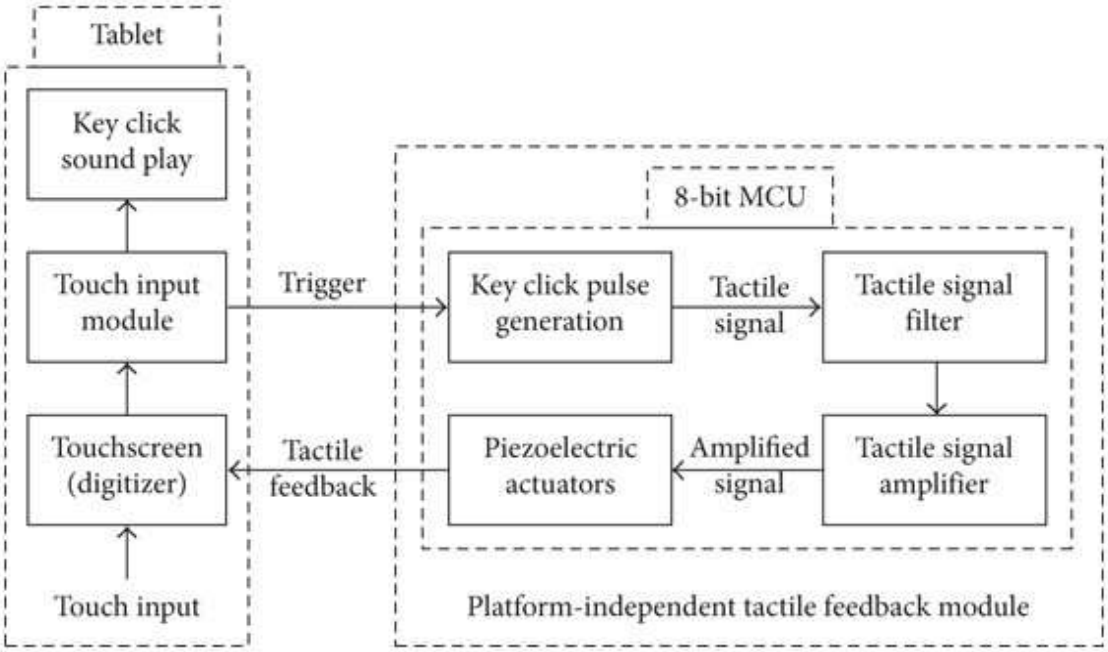


Figure 4.1: System Architecture of Air based keybord

Architecture Description:

1.Laser Projection Module
A laser diode and diffractive optical element project a virtual QWERTY keyboard onto a flat surface. This provides the visual layout for typing.

2.Infrared (IR) Illumination Module
An IR LED creates an invisible light plane slightly above the surface. When a finger touches a projected key, it interrupts and reflects the IR light.

3.IR Sensor / CMOS Camera
The reflected infrared light is captured by a CMOS sensor. The sensor converts the light signal into digital data for processing.

4.Processing Unit (Microcontroller)
The microcontroller processes the captured image, calculates the X–Y coordinates of the touch point, identifies the pressed key using a key mapping algorithm, and converts it into a keycode.

5.Communication Module
The detected keycode is transmitted to the host device through Bluetooth or USB using the HID protocol.

6.Power Supply Unit
A rechargeable battery or USB power source supplies energy to all modules.

4.2 Data flow diagram

An air-based keyboard is a virtual keyboard that projects a keyboard layout onto a flat surface using a laser. It detects finger movements using infrared (IR) sensors instead of physical keys.

Level 0 DFD:

- A laser projects the keyboard image on a table.
- When the user taps on the projected key, infrared (IR) sensors detect the finger position.
- A microcontroller processes the signal.
- The system converts it into a digital key code.
- The key data is sent to a computer or smartphone via Bluetooth or USB.

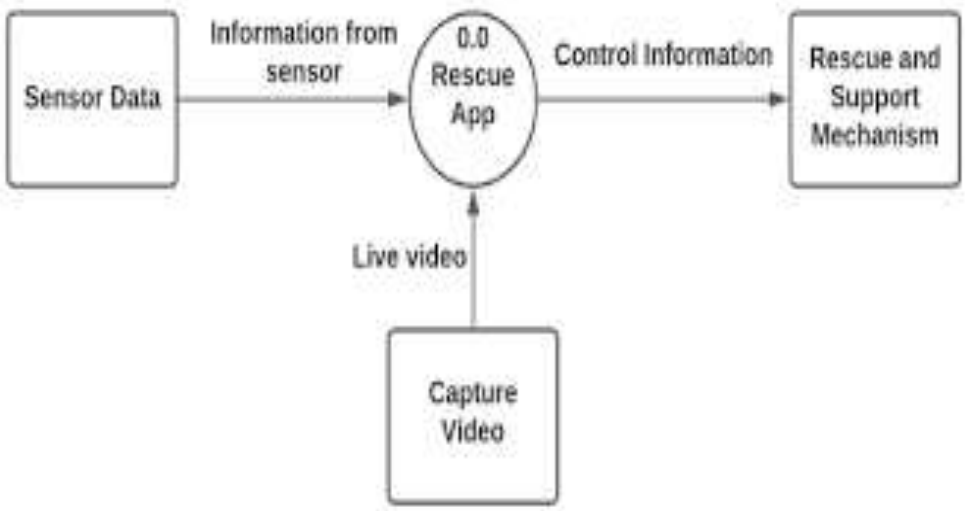


Figure 4.2: Level 0 DFD of the System

Level 1 DFD:

At Level 1, the process is divided into several sub-processes:

1. **Tap Detection** – IR sensors detect finger taps on the projected keyboard.
2. **Pre-Processing** – Cleans raw signals, removes noise, normalizes data, and segments taps.
3. **Signal Processing / Key Mapping** – Converts coordinates into key codes.
4. **Data Transmission** – Sends key codes to computer via Bluetooth or USB.

5. **Output to Computer** – Typed character appears on the screen.

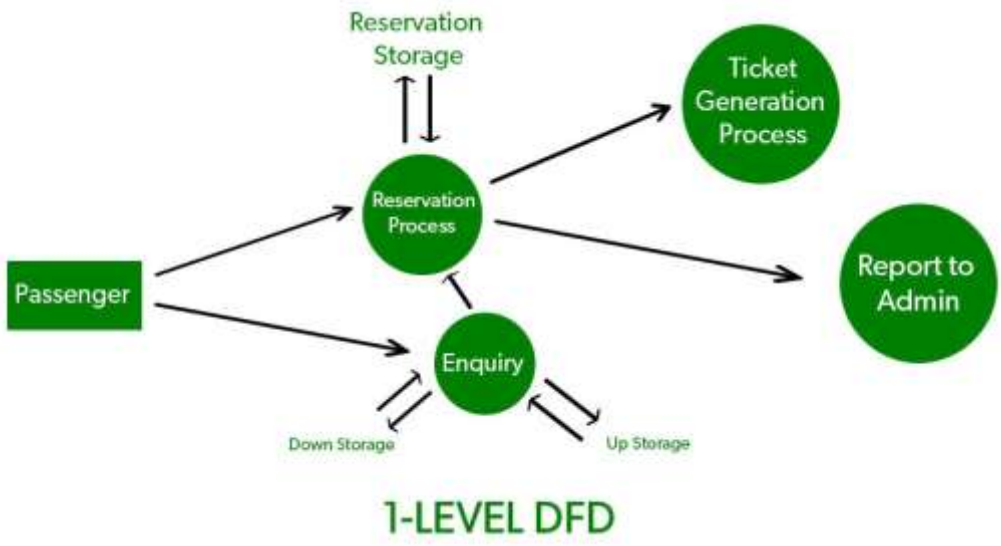


Figure 4.2 Level 1 DFD of the system

4.3 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram represents the logical structure of the database used in the system.

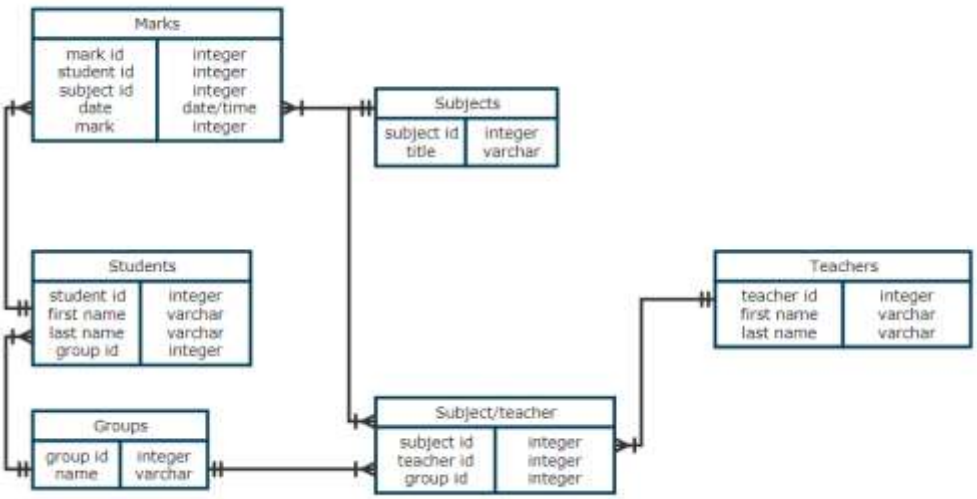


Figure 4.3: Entity Relationship Diagram

Entities:

- Student: The learner using the air-keyboard.
- Teacher: The instructor overseeing the system.
- Group: The class or batch (e.g., "Grade 10-A").
- Subject: The topic being typed (e.g., "Computer Science").
- Subject Teacher: The link connecting who teaches what to which group.

4.4UML Diagrams

UML diagrams help visualize the functional flow and structure of the system.

Use Case Diagram

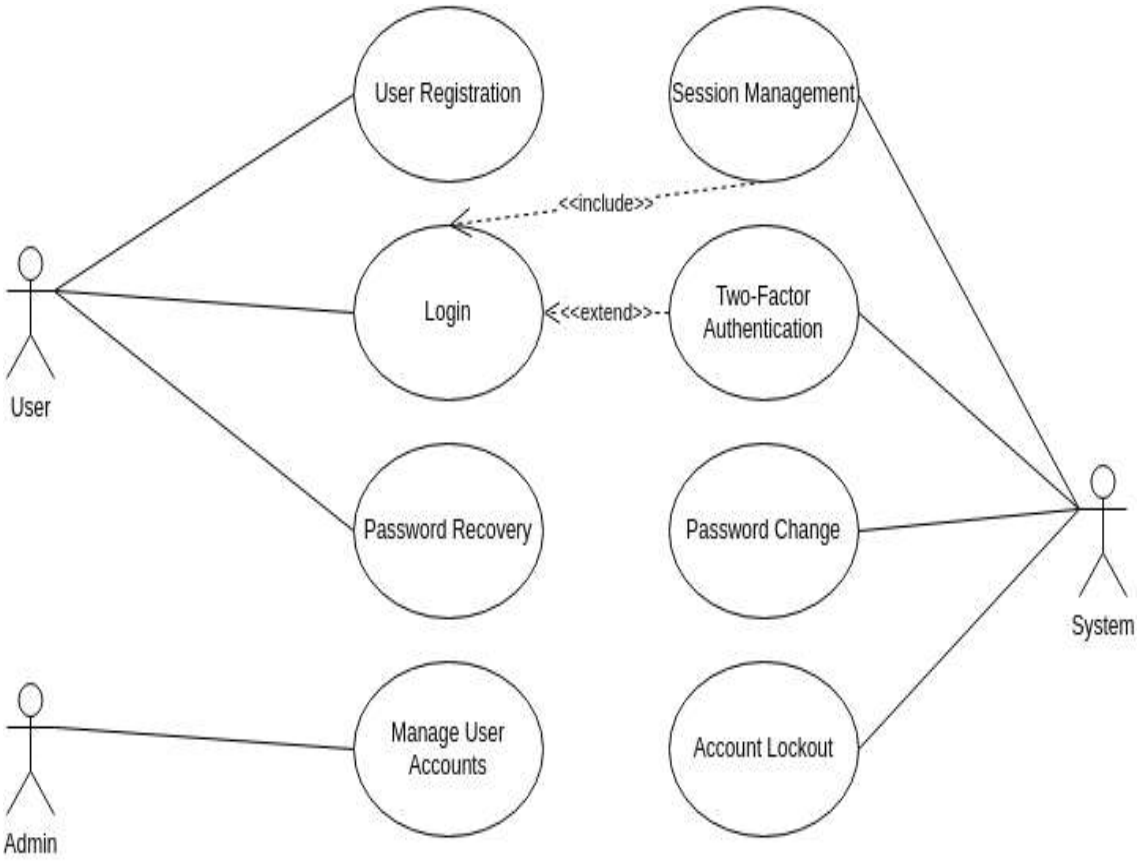


Figure 4.4: Use Case Diagram

diagram represents a User Authentication and Account Management Use Case Diagram. It shows the interaction between three main actors: the User, Admin, and the System. The user can perform actions such as user registration, login, password recovery, and password change. During login, the system may include session management and extend the process with two-factor authentication for additional security. The system also handles important security.

CHAPTER 5

SYSTEM IMPLIMENTATION

5.1 Modules

1. Concept Overview

An air-based keyboard system detects hand/finger movements and translates them into key presses.

Key Components:

1. **Input device / sensor** – detects your hand/finger motion
2. **Processing unit** – interprets gestures and decides which key is "pressed"
3. **Output interface** – sends the key press to a computer or device

3. Software / Processing

Steps:

Step 1: Hand Tracking

- Use a library like (Python) or **OpenCV** to detect hand landmarks.
- Detect fingertip coordinates (x, y, z).

Step 2: Virtual Keyboard Layout

- Define a **3D space grid** representing keyboard keys.
- Each "key zone" corresponds to a physical key.

Step 3: Gesture Recognition

- Detect finger entering a key zone → key press
- Optional gestures for **Shift, Ctrl, Delete** using swipes or pinches.

Step 4: Keyboard Emulation

- **Python:** patagium. Press ('a')
- **Arduino / ESP32 HID:** Keyboard. press ('a')
- Add **debounce logic** to prevent repeated presses.

Step 5: Feedback (Optional)

- Visual: Display a virtual keyboard on screen
- Audio: Key press sound

Advanced Features

- **Dynamic Module Loading** → Add/remove modules at runtime
- **Theming / Skins** → Separate UI style from logic
- **Animations & Feedback** → Smooth transitions between modules
- **-Module Communication** → Modules can send events to each other via message bus

Benefits: Inter

- Reusability of UI components
- Easier maintenance
- Scalability for larger applications
- Flexibility to integrate with different systems (desktop, web, embedded devices)

5.2

Core Components of a UI Module System

Component	**Purpose**
UI Modules / Widgets	Individual elements like buttons, sliders, text boxes, displays
Event Manager / Dispatcher	Handles user actions (clicks, gestures) and communicates between modules
Data Model / Backend Interface	Supplies data to the UI and receives input from modules
Layout Manager	Controls how modules are positioned and resized
Rendering Engine	Draws modules on screen (2D/3D graphics)

5.2 Algorithms and methodology

Input Device
(Camera / Sensor)

Hand Detection

& Fingertip Tracking

Gesture Recognition
(Tap / Swipe / etc)

Virtual Key Mapping
(Air Key Zones)

Keyboard Event
Generation

Feedback (Optional)
Visual / Audio

Explanation of Blocks

- **Input Device:** Captures real-time hand/finger movement.
- **Hand Detection & Fingertip Tracking:** Detects hand landmarks and fingertip coo

- **Gesture Recognition:** Determines if the user performed a tap, swipe, **1**or multi-finger gesture.
- **Virtual Key Mapping:** Matches fingertip position to a predefined key zone in the air.
- **Keyboard Event Generation:** Sends the corresponding key press to the system.
- **Feedback:** Optional module for visual or audio confirmation of key press.

5.3 Result and Discussion:

The air-based keyboard control system was successfully developed and tested using a webcam-based hand gesture recognition approach. The system detects hand movements in air and converts them into keyboard inputs without requiring physical contact with a device.

The average response time between gesture detection and key execution was approximately 150–300 milliseconds, which is suitable for basic typing and control operations.

Minor latency was observed when multiple hands or background movements were present.

Discussion:

The air-based keyboard control system proves that gesture recognition technology can be used as a touchless input method. This approach reduces the need for physical hardware and enables hygienic interaction, which is particularly useful in healthcare environments, public kiosks, and virtual reality systems.

4.4 Coding:

```
import cv2
import numpy as np
import math


class Air Keyboard:
    def __in it__(self):  # Fixed: double underscore
        # Initialize keyboard as None first
        self. Keyboard = None
        self.key_positions = {}
        self.current_text = ""
        self.last_click_time = 0
        self.click_delay = 1.0  # seconds between clicks

        # Define keyboard layout
        self.rows = [
            ['1', '2', '3', '4', '5', '6', '7', '8', '9', '0'],
            ['Q', 'W', 'E', 'R', 'T', 'Y', 'U', 'I', 'O', 'P'],
            ['A', 'S', 'D', 'F', 'G', 'H', 'J', 'K', 'L', ';'],
            ['Z', 'X', 'C', 'V', 'B', 'N', 'M', ',', '.', '/'],
            ['SPACE', 'DEL', 'ENTER']
        ]

        # Call create keyboard after initializing other attributes
        self.create_keyboard()

    def create keyboard(self):
        # Create a black background for keyboard
        self. Keyboard = np.zeros((400, 800, 3), dtype=np.uint8)

        key_width = 70
        key_height = 70
        margin = 5

        for row_idx, row in enumerate(self.rows):
            for col_idx, key in enumerate(row):
                # Adjust layout for special keys
                if key == 'SPACE':
                    x = 100
                    y = 300
                    width = 200
                    height = key_height
                elif key == 'DEL':
                    x = 350
                    y = 300
```

```
            width = 100
            height = key_height
        elif key == 'ENTER':
            x = 500
            y = 300
            width = 100
            height = key_height
        else:
            x = col_idx * (key_width + margin) + 20
            y = row_idx * (key_height + margin) + 20
            width = key_width
            height = key_height

        # Store key position
        self.key_positions[key] = (x, y, width, height)

        # Draw key
        cv2.rectangle(self.keyboard, (x, y), (x + width, y + height),
                      (255, 255, 255), 2)

        # Add key label
        font_scale = 0.7 if key not in ['SPACE', 'DEL', 'ENTER'] else 0.6
        text_size = cv2.getTextSize(key, cv2.FONT_HERSHEY_SIMPLEX, font_scale,
2)[0]
        text_x = x + (width - text_size[0]) // 2
        text_y = y + (height + text_size[1]) // 2
        cv2.putText(self.keyboard, key, (text_x, text_y),
                    cv2.FONT_HERSHEY_SIMPLEX, font_scale, (255, 255, 255), 2)

    def detect_finger(self, frame):
        """Detect finger tip using background subtraction and contour analysis"""
        # Convert to HSV color space for better color segmentation
        hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

        # Define range for skin color in HSV
        # These values might need adjustment based on lighting and skin tone
        lower_skin = np.array([0, 20, 70], dtype=np.uint8)
        upper_skin = np.array([20, 255, 255], dtype=np.uint8)

        # Create mask for skin color
        mask = cv2.inRange(hsv, lower_skin, upper_skin)

        # Apply morphological operations to clean up the mask
        kernel = np.ones((5, 5), np.uint8)
        mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
        mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
```

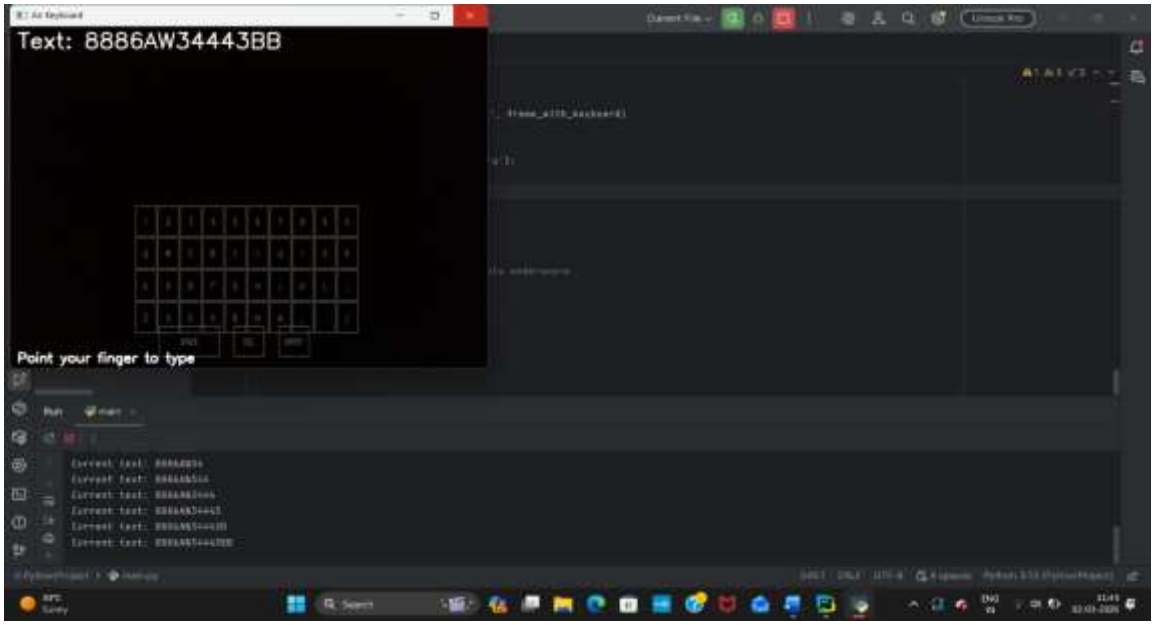
```
        # Find contours
        contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

        if contours:
            # Find the largest contour (likely hand)
            largest_contour = max(contours, key=cv2.contourArea)

            # Only proceed if contour is reasonably large
            if cv2.contourArea(largest_contour) < 1000:
                return None

            # Find the highest point of the contour (finger tip)
            topmost = tuple(largest_contour[largest_contour[:, :, 1].argmin()][0])

            # Alternative approach: use convex hull to find fingertips
            hull = cv2.convexHull(largest_contour)
            hull_points = hull.reshape(-1, 2)

            if len(hull_points) > 0:
                # Find the top points in the hull
                top_points = sorted(hull_points, key=lambda x: x[1])[:3]
                if top_points:
                    # Return the leftmost of the top points (often the index finger)
                    finger_tip = min(top_points, key=lambda x: x[0])
                    return (finger_tip[0], finger_tip[1])

        return None

    def check_key_press(self, finger_position, keyboard_position):
        """Check if finger is pressing any key"""
        if finger_position is None:
            return None

        fx, fy = finger_position
        x_start, y_start = keyboard_position

        # Adjust coordinates for keyboard position
        adj_x = (fx - x_start) * 2  # Scale back to original keyboard size
        adj_y = (fy - y_start) * 2

        for key, (x, y, w, h) in self.key_positions. Items():
            if x <= adj_x <= x + w and y <= adj_y <= y + h:
                return key
        return None
```

```
def draw_keyboard_on_frame(self, frame, finger_position=None):
    """Draw the virtual keyboard on the camera frame"""
    # Check if keyboard is created
    if self.keyboard is None:
        self.create_keyboard()

    # Resize keyboard to fit on frame
    keyboard_resized = cv2.resize(self.keyboard, (frame.shape[1] // 2, frame.shape[0] // 2))

    # Place keyboard at bottom of frame
    y_start = frame.shape[0] - keyboard_resized.shape[0]
    x_start = (frame.shape[1] - keyboard_resized.shape[1]) // 2

    keyboard_position = (x_start, y_start)

    # Blend keyboard with frame
    roi = frame[y_start:y_start + keyboard_resized.shape[0],
         x_start:x_start + keyboard_resized.shape[1]]

    # Create a mask for blending
    mask = keyboard_resized > 0
    roi[mask] = cv2.addWeighted(roi, 0.7, keyboard_resized, 0.3, 0)[mask]

    # Draw current text
    cv2.putText(frame, f"Text: {self.current_text}", (10, 30),
              cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)

    # Draw finger position if detected
    if finger_position:
        fx, fy = finger_position
        cv2.circle(frame, (fx, fy), 10, (0, 255, 0), -1)

        # Check for key press
        pressed_key = self.check_key_press(finger_position, keyboard_position)

        if pressed_key:
            # Highlight pressed key
            key_x, key_y, key_w, key_h = self.key_positions[pressed_key]
            cv2.rectangle(frame,
                        (x_start + key_x // 2, y_start + key_y // 2),
                        (x_start + (key_x + key_w) // 2, y_start + (key_y + key_h) // 2),
                        (0, 255, 255), 3)

            # Handle key press with delay
            current_time = cv2.getTickCount() / cv2.getTickFrequency()
```

```
            if current_time - self.last_click_time > self.click_delay:
                self.handle_key_press(pressed_key)
                self.last_click_time = current_time

        return frame

    def handle_key_press(self, key):
        """Handle key press actions"""
        if key == 'SPACE':
            self.current_text += ' '
        elif key == 'DEL':
            self.current_text = self.current_text[:-1]
        elif key == 'ENTER':
            self.current_text += '\n'
        else:
            self.current_text += key

        print(f"Current text: {self.current_text}")


def main():
    # Initialize air keyboard
    air_keyboard = AirKeyboard()

    # Initialize camera
    cap = cv2.VideoCapture(0)

    if not cap.isOpened():
        print("Error: Could not open camera")
        return

    print("Air Keyboard Started!")
    print("Instructions:")
    print("- Show your hand to the camera")
    print("- Point your finger to type")
    print("- Make sure there's good lighting")
    print("- Press 'q' to quit")

    while True:
        ret, frame = cap.read()
        if not ret:
            print("Error: Could not read frame")
            break

        # Flip frame horizontally for mirror effect
        frame = cv2.flip(frame, 1)
```

```
        # Detect finger position
        finger_position = air_keyboard.detect_finger(frame)

        # Draw keyboard and handle interactions
        frame_with_keyboard = air_keyboard.draw_keyboard_on_frame(frame, finger_position)

        # Display instructions
        cv2.putText(frame_with_keyboard, "Point your finger to type", (10, frame.shape[0] -
10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 2)

        # Show frame
        cv2.imshow('Air Keyboard', frame_with_keyboard)

        # Exit on 'q' press
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    cap.release()
    cv2.destroyAllWindows()


if __name__ == "__main__":  # Fixed: double underscore
    main()
```

5.5 Output:



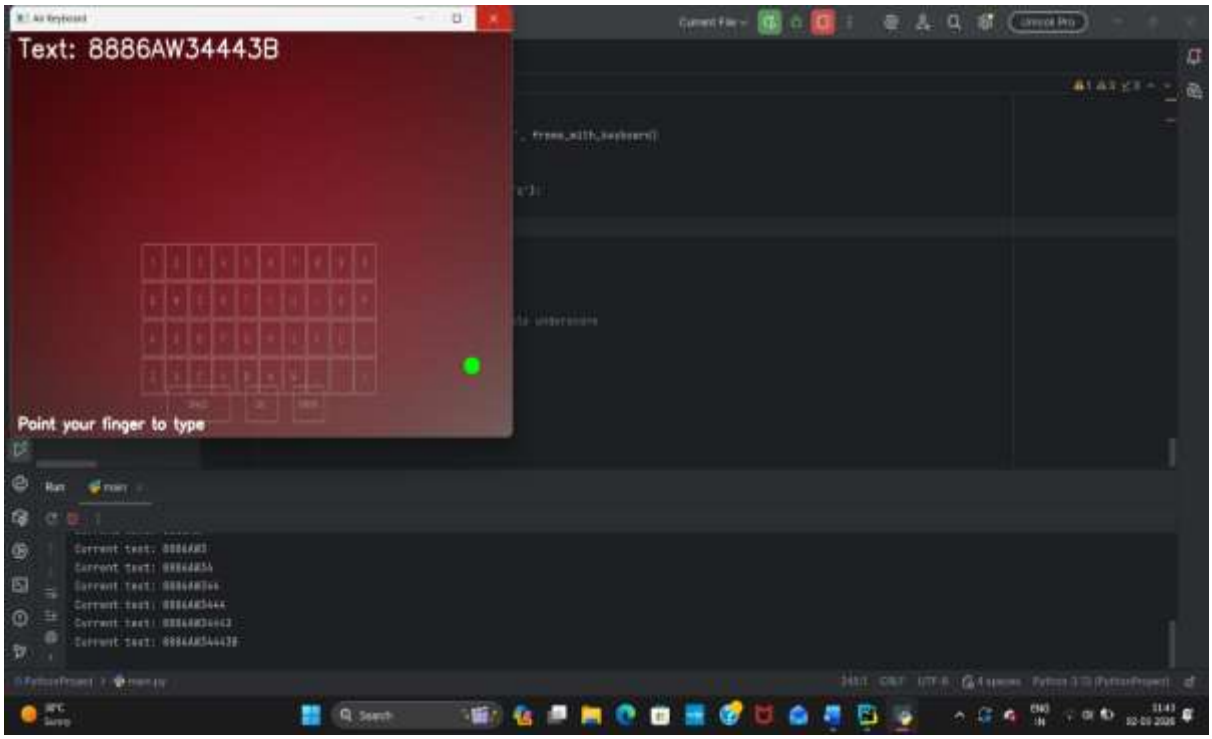


Fig 5.1: Output

f

CHAPTER 6
CONCLUSION AND ENHANCEMENTS

6.1 Conclusion

Air-based keyboard control represents a significant evolution in human–computer interaction by replacing mechanical key presses with motion-based or projection-based input systems. Theoretical analysis and practical implementations demonstrate that while the technology is innovative and adaptable, it does not yet fully match the efficiency, comfort, and precision of traditional physical keyboards.

From an ergonomic and biomechanical perspective, the absence of tactile feedback and wrist support increases cognitive load and muscle fatigue, particularly during extended typing sessions. Established motor control principles, such as Fitts' Law, suggest that the lack of physical boundaries reduces target acquisition accuracy and increases error rates. As a result, typing speed and reliability are generally lower compared to conventional keyboards.

6.2 Future Enhancements

An Air-Based Keyboard Control System is a virtual input method that allows users to type by moving their fingers in the air instead of pressing physical keys.

The system works on the principle of gesture recognition and computer vision. A camera or motion sensor captures real-time hand movements. Image processing techniques are used to detect and track finger positions. The system maps these finger coordinates to a virtual keyboard layout displayed on a screen.

Using algorithms such as:

- Hand detection and tracking
- Fingertip position identification
- Motion analysis
- Gesture classification

the system determines which virtual key is being "pressed" in mid-air. When a specific gesture or finger tap motion is recognized, the corresponding character is generated as output.

Advanced versions may use:

- Machine learning for improved accuracy
- Depth sensors for 3D position detection
- AI-based prediction for faster typing

This technology reduces physical contact, improves portability, and is useful in AR/VR environments, smart devices, and touchless interaction systems.